

Software Verification and Validation

Part Time Lecturer
Talimul Bari Shreshtho

Islamic University of Technology

June 12, 2026



Table of Contents

- 1 Myth 1
- 2 Myth 2
- 3 Myth 3
- 4 Myth 4
- 5 Myth 5
- 6 Myth 6
- 7 Myth 7
- 8 More Myths

Seven Myths of Formal Methods

Why Discuss Myths?

Formal Methods (FMs) are often misunderstood and surrounded by misconceptions. Many engineers believe they are:

- Too theoretical
- Too expensive
- Too difficult to learn
- Only useful for safety-critical systems

Reality:

- Formal methods are increasingly used in industry.
- They complement traditional testing and code reviews.
- Modern tools provide significant automation.
- They help detect defects early, when they are cheapest to fix.

Myth #1: Formal Methods Guarantee Perfect Software

The Myth

Formal methods can guarantee that software is perfect and completely free of defects.

Reality

- Nothing in engineering can guarantee perfection.
- Formal methods increase confidence in correctness.
- They can prove properties of models and specifications.
- They cannot guarantee that the real system behaves exactly as intended.

Key Takeaway

Formal methods reduce risk and defects, but they do not eliminate all possible errors.

Why Formal Methods Are Fallible

Two Fundamental Limitations

- ① Some properties cannot be proven.
- ② Humans can make mistakes in specifications and proofs.

A proof only shows that

Specification $\Rightarrow$ Property

It does NOT show that

- The specification is correct.
- The model matches the real world.
- The implementation faithfully represents the model.

Important

A verified wrong specification is still wrong.

Formal Models vs. The Real World

Consider a Civil Engineer

- Uses mathematical models to design a bridge or crane.
- Calculates loads, stress, and safety margins.
- Cannot mathematically prove that the real structure will never fail.

Similarly in Software

- Formal methods reason about models.
- Models are abstractions of reality.
- The real system may differ from the model.

Key Insight

Formal methods improve engineering rigour in software just as mathematics improves rigour in civil engineering.

Formal models do not capture everything

- Timing behavior and real-time constraints
- Performance characteristics
- Reliability and availability
- Human interaction and usability
- Hardware faults
- Operating system behavior
- Compiler and runtime issues

Conclusion

Formal verification proves properties of a model, not the entire real-world system.

If a formally verified program crashes because of a hardware fault, compiler bug, or incorrect requirement, did the formal verification fail?

Think and Discuss

Myth #2: Formal Methods are All About Program Verification

The Myth

Formal methods are mainly about proving that programs are correct.

Reality

- Formal methods are primarily about **specifications**.
- Program verification is only one activity.
- Most defects originate long before implementation.
- Finding requirement errors early is far cheaper than fixing code later.

Key Idea

Before proving a program is correct, we must define precisely what “correct” means.

Activities in Formal Methods

Formal methods apply mathematics throughout the software lifecycle.

- 1 Writing a formal specification
- 2 Proving properties of the specification
- 3 Refining specifications into designs
- 4 Constructing implementations
- 5 Verifying implementations

Requirements $\rightarrow$ Specification $\rightarrow$ Design $\rightarrow$ Code

Observation

Program verification is only one step in a much larger process.

Why Specifications Matter More

Bugs become more expensive to fix later.

Specification Error	\$1
Design Error	\$10
Implementation Error	\$100
Released Product Error	\$1000

Implication

- A correct program implementing the wrong requirements is still wrong.
- Early specification errors propagate throughout the system.
- Formal specifications help detect ambiguity and inconsistency before coding begins.

What Does a Formal Specification Provide?

A formal specification describes:

- What the system should do
- Constraints the system must satisfy
- Relationships among system entities
- Expected behavior of operations

It does not describe

- Programming language
- Data structures
- Algorithms
- User interface implementation

Key Insight

Specifications focus on **what** the system should do, not **how** it will do it.

Myth #3: Formal Methods Are Only Useful for Safety-Critical Systems

The Myth

Formal methods are only useful for aircraft, medical devices, nuclear plants, and other safety-critical systems.

Reality

- Formal methods help any system where failure is costly.
- Cost may involve:
 - Human safety
 - Financial loss
 - Security breaches
 - Loss of reputation
 - Operational disruption

Key Idea

Formal methods are valuable whenever correctness matters.

When Are Formal Methods Useful?

Formal methods are particularly useful when software is:

- Safety-critical
- Security-critical
- Financially critical
- Replicated millions of times
- Embedded in hardware
- Difficult or expensive to update

Examples

- Aircraft control software
- Banking systems
- E-commerce platforms
- Cloud services
- Operating systems
- Smart cards and payment systems

Benefits Beyond Program Verification

Even if no formal proof is performed, formal specifications can:

- Clarify requirements
- Remove ambiguity
- Improve communication with stakeholders
- Detect inconsistencies early
- Simplify maintenance
- Improve documentation quality

Observation

Many benefits of formal methods come from writing precise specifications.

Formality Is Not All-or-Nothing

Different projects require different levels of rigor.

Informal Requirements $\rightarrow$ Formal Specification $\rightarrow$ Verification $\rightarrow$ Fully
Formal Development

- Critical systems may require full verification.
- Many systems benefit from specifications alone.
- The degree of formality should match the project's needs.

Key Insight

You do not need full formal verification to gain value from formal methods.

Myth #4: Formal Methods Require Advanced Mathematics

The Myth

Formal methods can only be used by highly trained mathematicians.

Reality

- Most formal specifications use elementary mathematics.
- The primary foundations are:
 - Set Theory
 - Logic
 - Relations and Functions
- These concepts are typically taught in introductory discrete mathematics courses.

Key Idea

The mathematics is usually simpler than the software being specified.

Mathematics Used in Formal Specifications

Most specification languages rely on concepts you already know.

- Sets

$$Person = \{Alice, Bob, Carol\}$$

- Relations

$$Parent \subseteq Person \times Person$$

- Functions

$$spouse : Person \rightarrow Person$$

- Logic

$$\forall p : Person \mid p \notin p.^{children}$$

Observation

No calculus, differential equations, or advanced mathematics are required.

The Real Challenge

Formal methods are difficult because of modeling, not mathematics.

Common challenges:

- Choosing the right level of abstraction
- Understanding requirements
- Identifying important system properties
- Avoiding over-specification
- Avoiding under-specification

Example

- Too concrete: specifying database tables and algorithms
- Too abstract: omitting essential system constraints

Which Is Harder?

Programming

- Syntax
- APIs
- Libraries
- Concurrency
- Memory management
- Frameworks

Formal Specification

- Sets
- Relations
- Functions
- Logic
- Abstraction

Key Insight

Anyone who can learn a programming language can learn a specification language such as Alloy, Z, or VDM.

Myth #5: Formal Methods Increase Development Cost

The Myth

Formal methods make software development more expensive.

Reality

- Fully formal verification can be expensive.
- However, formal specifications often reduce overall project cost.
- Early defect detection is much cheaper than late defect correction.

Key Idea

Formal methods shift effort to the beginning of a project, where mistakes are cheapest to fix.

The Cost of Late Error Detection

Phase	Relative Cost
Requirements	1x
Design	5x–10x
Implementation	10x–50x
Testing	50x–100x
After Release	100x+

Observation

- Formal specifications uncover errors early.
- Fixing a requirement error is far cheaper than fixing deployed software.

How Formal Methods Change the Development Process

Phase	Traditional	Formal
Requirements	Low	High
Specification	Low	High
Implementation	High	Moderate
Testing	High	Lower
Maintenance	High	Lower

Observation

The total effort may remain similar or even decrease, but more effort is invested early in the lifecycle.

Choosing the Right Level of Formality

Not every project requires full formal verification.

- Small website → informal methods may be sufficient
- Banking software → formal specifications are valuable
- Aircraft control software → full verification may be justified

Key Insight

The level of formality should match the cost of failure.

Myth #6: Formal Methods Are Unacceptable to Users

The Myth

Users cannot understand or accept formal specifications.

Reality

- Formal specifications can help users understand requirements better.
- They describe **what the system should do** before implementation.
- When presented properly, they improve communication, not reduce it.

Key Idea

Formal specifications are not for replacing users—they are for communicating with users more precisely.

Making Formal Specifications User-Friendly

A formal specification should be made understandable using:

- **Natural language explanation**
 - Paraphrase mathematical definitions
- **Demonstration of consequences**
 - Show what the system guarantees
- **Animation / prototyping**
 - Visualize system behavior

Key Insight

The mathematics is for precision; the explanation is for humans.

Why Users Benefit from Formal Specifications

Well-written formal specifications can:

- Clarify unclear requirements
- Remove ambiguity
- Reveal inconsistencies early
- Provide a precise contract of system behavior
- Improve communication between users and developers

Example Insight

- Informal: "The system should be secure"
- Formal: "No unauthorised user can access confidential data"

Formal Methods and Prototyping are Complementary

- Prototypes help explore the user interface and behaviour
- Formal specifications capture system logic precisely

Effective Workflow

Prototype $\rightarrow$ Understanding $\rightarrow$ Formal Specification $\rightarrow$
Implementation

Key Idea

Prototyping explores ideas; formal methods lock them down precisely.

Myth #7: Formal Methods Are Not Used in Real Systems

The Myth

Formal methods are only academic and not used in large-scale industrial software.

Reality

- Formal methods are actively used in industry.
- They appear in both software and hardware systems.
- They are applied in safety, security, and business-critical domains.

Key Idea

Formal methods are not theoretical—they are industrial engineering tools.

Where Formal Methods Are Used

Formal methods have been applied in many real-world domains:

- **Transaction systems**
 - IBM CICS (large-scale transaction processing systems)
- **Hardware design**
 - Floating-point units
 - Secure processor architectures
- **Compilers**
 - Industrial compiler development
- **Software tools**
 - CASE tools and software engineering environments

Why Industry Uses Formal Methods

Formal methods provide real engineering benefits:

- Early detection of design errors
- Improved system reliability
- Better understanding of requirements
- Support for maintenance and evolution
- Increased confidence in critical systems

Key Insight

Formal methods are most valuable when failure is expensive.

Formal Methods Are Not a Silver Bullet

- Formal methods do not eliminate all software problems.
- They are part of a larger engineering process.
- They must be combined with:
 - Testing
 - Design techniques
 - Project management
 - Quality assurance

Reality Check

Formal methods improve development—they do not replace engineering discipline.

Seven Facts About Formal Methods

Instead of perpetuating myths, we summarize seven key facts:

- ➊ Formal methods are very helpful at finding errors early on and can nearly eliminate certain classes of errors.
- ➋ They work largely by making you think very hard about the system you propose to build.
- ➌ They are useful for almost any application domain.
- ➍ They are based on mathematical specifications, which are much easier to understand than programs.
- ➎ They can decrease the cost of software development.
- ➏ They can help clients understand what they are buying.
- ➐ They are being used successfully on practical industrial projects.

Key Message

Formal methods are practical engineering tools that improve correctness, clarity, and communication in software development.

Myth: Formal Methods Delay Development

The Myth

Formal methods slow down the software development process.

Fact

- The Inmos T800 floating-point unit (Z + Occam Transformation System) was completed **12 months ahead of schedule**.
- IBM's CICS system using Z reported about **9% reduction in development cost**.

Key Insight

Better specification often leads to faster and more predictable development.

Myth: Formal Methods Lack Tools

The Myth

Formal methods are difficult to use because tools are unavailable.

Fact

- Many mature tools exist:
 - Z notation tools
 - Alloy Analyzer
 - TLA / TLA+
- Modern model checkers and SMT solvers support automation.

Key Insight

Formal methods are tool-supported and increasingly automated.

Myth: Formal Methods Replace Engineering Design

The Myth

Formal methods replace traditional software engineering practices.

Fact

- Formal methods are **not standalone methods**.
- They are formal mathematical systems used within engineering processes.
- They complement design, testing, and implementation.

Key Insight

Formal methods enhance engineering—they do not replace it.

Myth: Formal Methods Only Apply to Software

The Myth

Formal methods are only useful for software systems.

Fact

- Formal methods apply to **hardware and software**.
- Used in microprocessor and system verification.
- Example: Viper microprocessor verified using HOL theorem prover.

Key Insight

Formal reasoning applies to any system with precise behavior.

Myth: Formal Methods Are Unnecessary

The Myth

Formal methods are overkill and not needed in practice.

Fact

- Sometimes formal methods are overkill for small systems.
- For critical systems, they are highly valuable.
- Recommended whenever correctness, safety, or security matters.

Key Insight

Use formal methods where correctness matters most.

Myth: Formal Methods Are Not Supported

The Myth

Formal methods are not widely supported in education or industry.

Fact

- Formal methods are widely taught in universities.
- Common methods include:
 - Z
 - VDM
 - CSP
 - CCS
- Strong academic and industrial research ecosystem exists.

Key Insight

Formal methods are a mature academic and industrial discipline.

Myth: Formal-Methods Experts Always Use Formal Methods

The Myth

People trained in formal methods always use them for everything.

Fact

- Formal methods are used selectively, not universally.
- Engineers choose methods based on:
 - Risk level
 - Cost
 - System complexity
- Practical engineering always involves trade-offs.

Key Insight

Formal methods are tools—good engineers choose when to use them.

Thank You

Thank you for your attention!

`talimulbari@iut-dhaka.edu`